

From Reference Architecture to Living Ontology: the hyper-ontology/1 Package and Hyper Ontology

Every design Praxis produces ends in an object model. This paper specifies the package that carries it, measures the seven published packages, shows how to load them, and describes how Hyper Ontology turns one into a living system.

CodeNinja Engineering Team · Umar Bilal

CodeNinja · 2026-10-04 · CC BY 4.0 · Web edition: <https://muhammadumar89.github.io/codeninja-research/ontology-method/>

Abstract. Ontologies are to agents what databases are to humans: a database assumes a reader who already knows what the columns mean, while an ontology writes that meaning down once, as objects, properties, typed links and actions, so an agent resolves instead of guessing. Every system design reasoned on Praxis ends in such a model, packaged as `hyper-ontology/1`: typed objects with properties and status vocabularies, the system of record each is anchored in, typed directional links, the write paths and the human approval the design requires. Seven packages are published with the Vertical-Driven Architectures series, holding 94 objects and 86 typed links across 9 object kinds. An open, dependency-free loader validates them and converts them to Mermaid, Cypher and JSON-LD. Hyper Ontology, CodeNinja's ontology platform, imports a package and stands it up as a projection over the operator's own systems of record, connected through adapters that emit ontology events, with actions that write back under approval, so the model senses, decides, acts and learns.

1. Why the design ends in an ontology

A physical operation's meaning is scattered: the berth window lives in the port community system, the channel depth in the survey archive, the lease in the finance backbone. Every report, application and agent that reads the silos directly re-derives the business, and derives it

slightly differently. A design that ends in an object model fixes the meaning once. Praxis therefore treats the object model as the join the systems never made, and every paper in the series has a chapter that names each object, its anchor and its links.

2. The package

```
{
  "package": "hyper-ontology/1",
  "designed_with": "Praxis",
  "implemented_with": "Hyper Ontology",
  "paper": {
    "title": "...",
    "url": "...",
    "doi": "...",
    "sector": "...",
    "country": "...",
    "objects": [
      {
        "id": "berth",
        "label": "Berth",
        "kind": "asset",
        "anchored_in": "Port Community System",
        "properties": [
          "Berth ID",
          "Alongside depth",
          "Berth window",
          "status_vocabulary": [
            "Occupied",
            "Reserved",
            "Available"
          ],
          "links": [
            {
              "to": "navigation_channel",
              "label": "adjoins"
            }
          ]
        },
        "write_paths": [
          "...",
          "human_loop": "..."
        ]
      }
    ]
  }
}
```

Objects carry an id, a label and one of ten kinds: asset, record, event, person, actor, site, material, measure, document and system. **anchored_in** names the system of record the object is read from, or says the object is born in the design itself, such as a decision record. **Links** are typed and directional; the reverse is implied. A link whose label a paper's figure could not print carries a *note* instead of an invented label. **write_paths** and **human_loop** carry the design's rules about what may change the world and who approves it. Nothing in a package names the operator; the same gate that reads the paper reads the package. The full specification is [ONTOLOGY_PACKAGE.md](#).

3. Seven packages in numbers

DESIGN	COUNTRY	OBJECTS	LINKS	SYSTEMS ANCHORED	MOST-LINKED OBJECT
Sovereign HSE Watch	Pakistan	12	14	8	Operating Facility / Site (4 links)
Feeder Firewatch	United States	14	12	10	Feeder segment (6 links)
Terminal Pulse	United States	12	11	7	Container (6 links)
Structure Phase Watch	Saudi Arabia	15	12	10	Pour (4 links)
Steel Count Ledger	Pakistan	14	12	5	Industrial PC (4 links)
Factory Fire Watch	Saudi Arabia	14	12	8	Safety-critical alert (4 links)
Port Twin	United States	13	13	11	Berth (8 links)

Table 1. The seven published packages. Systems anchored counts the distinct systems of record the objects are read from.

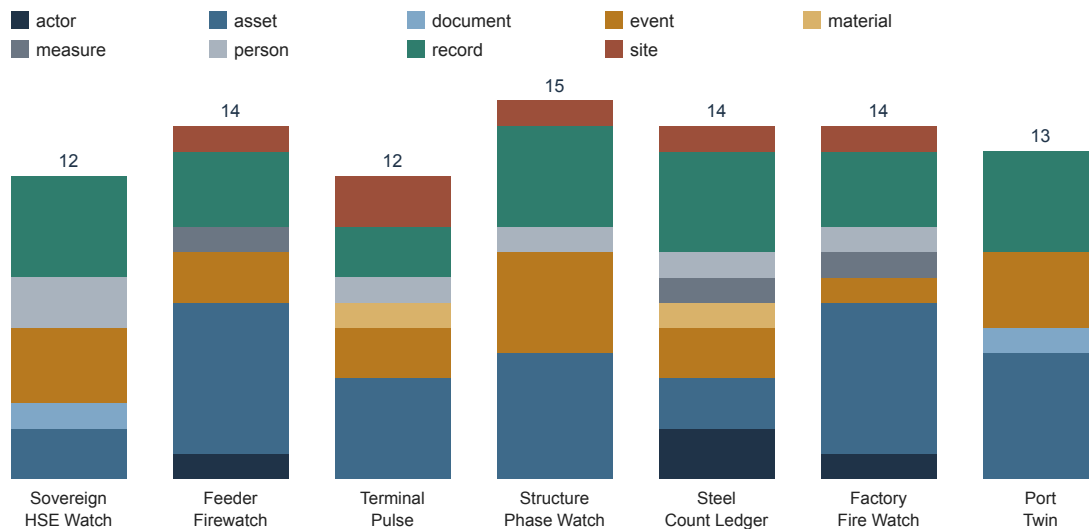


Figure 1. Objects per design by kind. Assets and records dominate, and every design carries at least one event: a moment that happens and that the model must record.

Across the series: 30 asset, 24 record, 17 event, 6 person, 6 site, 4 actor, 3 measure, 2 document, 2 material. The most-linked object is where the design's questions meet: the berth in

the port twin, the feeder segment in the wildfire design, the container in the terminal design, the pour on the construction site.

4. Reach: what a query on the model can traverse

The value of an ontology is the traversal a document store cannot make. From one berth in Port Twin, two hops of typed links reach:

```
berth ->[adjoins] navigation_channel
berth ->[receives] vessel_movement
berth ->[belongs to] parcel_facility
berth <-[monitors] environmental_sensor_feed
berth <-[linked] utility_gap_record
berth <-[linked] capital_project
berth <-[linked] inspection_record
berth <-[arrives via] pcs_message
berth ->[adjoins] navigation_channel ->[surveyed by] bathymetric_survey
berth ->[belongs to] parcel_facility ->[hosts] capital_project
```

From one production count event in Steel Count Ledger:

```
count_event <-[emits] ipc
count_event ->[classifies as] product_type
count_event <-[validates] calibration
count_event <-[emits] ipc <-[runs on] installation_point
count_event <-[emits] ipc ->[linked] uptime_record
count_event <-[emits] ipc ->[linked] tamper_alert
count_event ->[classifies as] product_type ->[linked] declared_production
count_event <-[validates] calibration ->[measured against] weighbridge_record
```

Both listings are the output of `hyper-ontology reach` on the published packages.

5. Loading a package

```
pip install "git+https://github.com/muhammadumar89/codeninja-
research#subdirectory=hyper-ontology-py"
hyper-ontology list
hyper-ontology show port-digital-twin-us
hyper-ontology validate steel-production-count-pakistan
hyper-ontology cypher truck-turn-container-terminal-us > load.cypher
hyper-ontology mermaid factory-fire-monitoring-saudi-arabia > model.mmd
```

The loader has no dependencies, validates kinds, ids and link targets, and converts a package to a Mermaid class diagram, a Cypher script for a graph database, or JSON-LD for a triple store. All seven published packages validate. The loader reads and converts the schema; it does not connect to live systems.

6. What Hyper Ontology adds

A package is the schema of a design. Hyper Ontology makes it living. It stands the model up in three layers, and the order is the argument: systems of record below, never replaced, modified or migrated; the ontology in the middle, built once and owned by the operator, as a projection over those systems rather than a copy; applications and agents on top, reading meaning from the model and never from the silos directly. Each anchored system connects through an adapter that emits ontology events, so the integration contract is the event, not a vendor's payload, and a system can be replaced without touching anything above the adapter.

The model has a grammar of four verbs: nouns become objects, facts become properties, relationships become typed links, and actions change them. The fourth is what makes the model living: it senses as data lands on objects, decides as questions are answered on the model, acts as approved decisions write back, and learns as the outcome lands on the model again. The package's write paths and human loop become the rules on those actions: a recommendation is a record, and a named person's approval is what turns it into a change.

7. Limits

A package is schema, not data: it names what the model holds and how it links, not the records of any operator. Status vocabularies a design left at a default were dropped rather than invented, and links a figure could not label are kept with a note. Hyper Ontology is in beta and used in house by CodeNinja; access for outside teams is by request at <https://muhammadumar89.github.io/codeninja-research/hyper-ontology/>.

References

1. CodeNinja Engineering Team and Umar Bilal. 2026. *Sovereign HSE Watch: Predictive Risk and Early Warning on an HSE Control and Command Platform*. CodeNinja. <https://doi.org/10.5281/zenodo.23119714>
2. CodeNinja Engineering Team and Umar Bilal. 2026. *Feeder Firewatch: Live Ignition and Outage Risk for Every Distribution Feeder*. CodeNinja. <https://doi.org/10.5281/zenodo.23119325>
3. CodeNinja Engineering Team and Umar Bilal. 2026. *Terminal Pulse: Predicted Truck Turn Time and Live Yard Sight for a Container Terminal*. CodeNinja. <https://doi.org/10.5281/zenodo.23119348>

4. CodeNinja Engineering Team and Umar Bilal. 2026. *Structure Phase Watch: Live Production, Crane and Delivery Evidence for Every Pour on a Construction Site*. CodeNinja. <https://doi.org/10.5281/zenodo.23126448>
5. CodeNinja Engineering Team and Umar Bilal. 2026. *Steel Count Ledger: Independently Counted Production for Every Steel Mill in Pakistan*. CodeNinja. <https://doi.org/10.5281/zenodo.23126563>
6. CodeNinja Engineering Team and Umar Bilal. 2026. *Factory Fire Watch: Read-Only Smart Fire Protection Monitoring for Every High-Risk Factory*. CodeNinja. <https://doi.org/10.5281/zenodo.23126565>
7. CodeNinja Engineering Team and Umar Bilal. 2026. *Port Twin: One Governed Digital Twin for Every Asset, Feed and Dollar*. CodeNinja. <https://doi.org/10.5281/zenodo.23126431>